

AI110 | Foundations of AI Engineering

Foundations of AI Engineering Summer 2026 (@ Section 1A | Tuesday 5PM - 7PM PT)
Personal Member ID#: **151098**

Welcome

Getting Started

► Course Info

AI Prompting Guide

Course Progress

► Module 1

► Module 2

► Module 3

▼ Module 4

Week 7

Week 8

► Module 5

► Project Grading

Overview

Tinker (Lab)

Show (Project)

 Reminder: Project 4 is due by **Monday, August 3rd at 1:59AM CDT.**

Show What You Know: Applied AI System

Project Overview

~4 hours

This final project is your opportunity to bring everything together, from debugging and design (Module 1-2) to reasoning and retrieval (Module 3-4), and finally agentic workflows and reliability testing (Module 5).

You'll **choose one of your previous projects** from Modules 1-3 and **extend it into a full applied AI system** that solves a meaningful problem or automates a reasoning task. This is your chance to evolve an earlier prototype into a polished, professional artifact.

Your system should demonstrate **responsible design**, **technical creativity**, and **clear explanation** of how the AI works and why it's trustworthy.

Goals

By completing this project, you will be able to...

- ☐ Extend and redesign a prior mini-project into a cohesive, end-to-end AI integrated system.
- ☐ Implement modular components (retrieval, logic, or agentic planning) using Python.
- ☐ Test and evaluate system reliability and guardrails through structured experiments.
- ☐ Document and explain the AI's decision-making process clearly and responsibly.
- ☐ Communicate results through a professional presentation and portfolio entry.

 Before you start, review the [Grading Rubric](#) to understand exactly how your project will be evaluated, including required features and optional stretch challenges worth extra points.

Project Requirements

Your final project is about taking what you've already built and making it stronger, smarter, and more reliable. You'll extend one of your previous projects (from Modules 1-3) into a complete applied AI system.

► Example Extensions

▼ 0. Preparing Your Project Environment

Because this project is an evolution of your earlier work, you should create a dedicated copy of your chosen repo. This allows you to experiment freely without altering your original Module 1-3 submission.

- ☐ Go to GitHub and create a new, public repo.
 - ☐ Name it something professional, like `applied-ai-system-project`.
 - ☐ Do **not** initialize it with a README, license, or `.gitignore`. Keep it completely empty.
 - ☐ Copy the URL of this new repo (e.g.,
`https://github.com/username/applied-ai-system-final.git`)
- ☐ Clone your original repo into a new folder on your computer:
 - ☐ `git clone https://github.com/username/your-original-repo.git applied-ai-system-final`
 - ☐ Enter the folder: `cd applied-ai-system-final`
- ☐ Update the remote to point to your new repo:
 - ☐ `git remote set-url origin https://github.com/username/applied-ai-system-final.git`
- ☐ Push everything to your new repo:
 - ☐ `git push -u origin main`

Note: If your default branch is named `master` instead of `main`, use
`git push -u origin master`.

- ☐ Verify the connection: go to your new repo on GitHub and confirm your code and commit history are there.

Before you begin coding, set up a professional folder structure in your new repo.

- ☐ Create a folder in your project called `assets`. This will be your dedicated directory for your system architecture images.
- ☐ Create a folder called `diagrams`. Your architecture diagram **must** be saved as a Mermaid source file (e.g., `diagrams/architecture.mmd`) — a `.mmd` file or a `.md` file containing a Mermaid code block. A PNG image alone is not sufficient.
- ☐ If you are using the Mermaid Live Editor to preview your diagram, copy the source text into your `.mmd` file before exporting. You may also export a PNG for embedding in the README, but the source file is required.

Close Section

▼ 1. Functionality: What Your System Should Do

Your project should **do something useful with AI**. For example:

- Summarize text or documents
- Retrieve information or data from a source
- Plan and complete a step-by-step task
- Help debug, classify, or explain something

To make your project more advanced, it must include **at least one** of the following AI features:

Feature	What It Means	Example
Retrieval-Augmented Generation (RAG)	Your AI looks up or retrieves information before answering.	A study bot that searches notes before generating a quiz question.
Agentic Workflow	Your AI can plan, act, and check its own work.	A coding assistant that writes, tests, and then fixes code automatically.
Fine-Tuned or Specialized Model	You use a model that's been trained or adjusted for a specific task.	A chatbot tuned to respond in a company's tone of voice.
Reliability or Testing System	You include ways to measure or test how well your AI performs.	A script that checks if your AI gives consistent answers.

The feature should be fully integrated into the main application logic. It is not enough to have standalone script; the feature must meaningfully change how the system behaves or processes information. For example, if you add RAG, your AI should actively use the retrieved data to formulate its response rather than just printing the data alongside a standard answer.

Also, make sure your project:

- **Runs correctly and reproducibly:** If someone follows your instructions, it should work.
- **Includes logging or guardrails:** Your code should track what it does and handle errors safely.
- **Has clear setup steps:** Someone else should be able to run it without guessing what to install.

Close Section

▼ 2. Design and Architecture: How Your System Fits Together

Show how your project is organized by creating a short **system diagram**. Your diagram should include:

- ☐ The **main components** (like retriever, agent, evaluator, or tester).
- ☐ How **data flows** through the system (input → process → output).
- ☐ Where **humans or testing** are involved in checking AI results.

Submit your diagram as a Mermaid source file (`.mmd`) or a Markdown (`.md`) file containing Mermaid syntax — do not submit it only as a screenshot or image. This lets the diagram's structure be read directly from the source.

Close Section

▼ 3. Documentation: How You Explain Your Work

You'll write a **README file** that clearly explains your project. It should include:

- ☐ Explicitly name your original project (from Modules 1-3) and provide a 2-3 sentence summary of its original goals and capabilities.
- ☐ **Title and Summary:** What your project does and why it matters.
- ☐ **Architecture Overview:** A short explanation of your system diagram.
- ☐ **Setup Instructions:** Step-by-step directions to run your code.
- ☐ **Sample Interactions:** Include at least 2-3 examples of inputs and the resulting AI outputs to demonstrate the system is functional.
- ☐ **Design Decisions:** Why you built it this way, and what trade-offs you made.
- ☐ **Testing Summary:** What worked, what didn't, and what you learned.
- ☐ **Reflection:** A brief note on what this project taught you about AI and problem-solving. Your **graded** responsible-AI reflection — how you collaborated with AI, one helpful and one flawed AI suggestion, and your system's limitations — goes in `model_card.md` (see Step 5), **not** here. Reflection content placed only in the README does not earn the reflection points.

Write this for a future employer who might look at your GitHub portfolio! Clarity and completeness matter more than perfection.

Close Section



4. Reliability and Evaluation: How You Test and Improve Your AI

Your AI should **prove that it works**, not just seem like it does. Include **at least one way** to test or measure its reliability, such as:

- Automated tests (e.g., unit tests or simple checks for key functions).
- Confidence scoring (the AI rates how sure it is).
- Logging and error handling (your code records what failed and why).
- Human evaluation (you or a peer review the AI's output).

Summarize your testing in a few lines, like:

5 out of 6 tests passed; the AI struggled when context was missing. Confidence scores averaged 0.8; accuracy improved after adding validation rules.

If you use human evaluation, document it in a parseable format — a markdown table or structured text/JSON — so your results can be read without watching a demo. For example:

Test Input	Evaluation Criteria	Result
"Summarize this article"	Accurate, under 100 words	Pass
Empty input	Handles gracefully	Fail — crashed

► 5. Reflection and Ethics: Thinking Critically About Your AI

► Optional: Stretch Features for Extra Points



6. Presentation and Portfolio: Sharing Your Work Professionally



Submitting Your Project

Once you've completed all the required features for your project, use the following checklist to prepare your work for submission.

1. Code is pushed to the correct GitHub repository

2. Repo is public
3. **Required files are present:** You must include your functional code, a comprehensive `README.md`, a `model_card.md` for reflections, and a System Architecture Diagram saved as a Mermaid source file (`.mmd` or `.md` with Mermaid code block) in a `diagrams/` folder.
4. **Reproducible Execution Evidence:** Instead of demo screenshots, include reproducible command outputs or interaction logs in markdown or `.txt` files. Your diagram source file and any optional supplementary images should be stored in dedicated `/assets` or `/diagrams` folders.
5. Commit history shows multiple meaningful commits
6. **Standardized Documentation:** Your `README.md` must identify your base project, and your `model_card.md` must answer all reflection prompts regarding AI collaboration, biases, and testing results.
7. **Demo evidence is included:** Your `README.md` contains at least 2-3 example inputs with their outputs pasted as fenced code blocks that demonstrate your system's behavior without relying on a video. A Loom walkthrough is optional but helpful for human reviewers; if you include one, add the link to your README.
8. Final changes are committed and pushed before the deadline

How It's Graded

A detailed breakdown of graded features and points can be found on the course [grading](#) page.

Close Section